

As you given previously do same for me for ETL Use case

Here is the step-by-step execution plan for **Use Case 6: Telecom Customer Usage ETL Pipeline using Airflow, HDFS, and Hive**, organized by terminal and process context.

TXT

All system-dependent parameters (paths, usernames, database names, table names, file names, hostnames, and ports) that need to be customized are enclosed in `<< >>`.

Group A: Environment & HDFS Data Ingestion

Terminal 1: Hadoop & Local Directory Setup

1. Verify Hadoop & HDFS Services:

Bash



```
jps  
hdfs dfs -ls /
```

1. Create Local Project Directory:

Bash



```
mkdir -p ~/<<telecom-usage-etl>>/<<data>>  
cd ~/<<telecom-usage-etl>>/<<data>>
```

1. Create the Input CSV File:

Bash



```
nano <<telecom_usage.csv>>
```

Paste Content:

TXT

Code snippet



```
<<usage_id>>,<<customer_id>>,<<plan>>,<<call_minutes>>,<<sms_count>>,<<data_gb>>,<<us
<<U001>>,<<C101>>,<<Premium>>,<<450>>,<<120>>,<<8.5>>,<<2026-09-25>>,<<Bangalore>>
<<U002>>,<<C102>>,<<Basic>>,<<120>>,<<45>>,<<2.3>>,<<2026-09-25>>,<<Mysore>>
<<U003>>,<<C103>>,<<Premium>>,<<620>>,<<180>>,<<15.2>>,<<2026-09-25>>,<<Bangalore>>
<<U004>>,<<C104>>,<<Basic>>,<<210>>,<<60>>,<<4.5>>,<<2026-09-26>>,<<Chennai>>
<<U005>>,<<C105>>,<<Premium>>,<<580>>,<<150>>,<<10.5>>,<<2026-09-26>>,<<Hyderabad>>
<<U006>>,<<C106>>,<<Basic>>,<<95>>,<<30>>,<<1.8>>,<<2026-09-26>>,<<Mangalore>>
<<U007>>,<<C107>>,<<Premium>>,<<710>>,<<220>>,<<18.7>>,<<2026-09-27>>,<<Bangalore>>
<<U008>>,<<C108>>,<<Basic>>,<<165>>,<<55>>,<<6.2>>,<<2026-09-27>>,<<Mysore>>
<<U009>>,<<C109>>,<<Premium>>,<<490>>,<<135>>,<<9.9>>,<<2026-09-27>>,<<Chennai>>
<<U010>>,<<C110>>,<<Basic>>,<<250>>,<<75>>,<<10.0>>,<<2026-09-28>>,<<Hyderabad>>
<<U011>>,<<C111>>,<<Premium>>,<<530>>,<<160>>,<<12.4>>,<<2026-09-28>>,<<Bangalore>>
<<U012>>,<<C112>>,<<Basic>>,<<140>>,<<40>>,<<3.7>>,<<2026-09-28>>,<<Mangalore>>
<<U013>>,<<C113>>,<<Premium>>,<<680>>,<<200>>,<<20.1>>,<<2026-09-29>>,<<Chennai>>
<<U014>>,<<C114>>,<<Basic>>,<<190>>,<<65>>,<<7.5>>,<<2026-09-29>>,<<Mysore>>
<<U015>>,<<C115>>,<<Premium>>,<<455>>,<<110>>,<<9.5>>,<<2026-09-29>>,<<Hyderabad>>
```

1. Verify Local CSV File:

Bash



```
cat <<telecom_usage.csv>>
```

Group B: Airflow DAG Orchestration Setup

Terminal 2: Airflow Environment & DAG Definition

1. Navigate to the Airflow DAGs Folder:

Bash



```
cd ~/airflow/dags
```

(If the folder does not exist, create it via `mkdir -p ~/airflow/dags`)

1. Create the Airflow DAG File:

Bash



```
nano <<telecom_etl_dag.py>>
```

1. Paste DAG Definition:

Python



```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG(
    dag_id="<<telecom_customer_usage_etl>>",
    start_date=datetime(2026, 10, 1),
    schedule=None,
    catchup=False,
) as dag:

    # 1. EXTRACT TASK
    extract = BashOperator(
        task_id="extract",
        bash_command="""
            test -f /home/<<ubuntu>>/<<telecom-usage-etl>>/data/<<telecom_usage.csv>>
            echo "Input CSV file found successfully"
            """,
    )

    # 2. LOAD RAW DATA INTO HDFS
    load_to_hdfs = BashOperator(
        task_id="load_to_hdfs",
        bash_command="""
```

```

hdfs dfs -mkdir -p /<<telecom>>/raw/<<customer_usage>>

hdfs dfs -put -f \
/home/<<ubuntu>>/<<telecom-usage-etl>>/data/<<telecom_usage.csv>> \
/<<telecom>>/raw/<<customer_usage>>/

echo "CSV loaded into HDFS successfully"

hdfs dfs -ls /<<telecom>>/raw/<<customer_usage>>
""",
)

# 3. TRANSFORM DATA USING HIVE
transform_hive = BashOperator(
    task_id="transform_hive",
    bash_command="""
beeline -u jdbc:hive2://<<localhost>>:<<10000>>/<<default>> -e "

CREATE DATABASE IF NOT EXISTS <<telecom_dw>>;

USE <<telecom_dw>>;

DROP TABLE IF EXISTS <<telecom_usage_raw>>;

CREATE TABLE <<telecom_usage_raw>> (
    usage_id STRING,
    customer_id STRING,
    plan STRING,
    call_minutes INT,
    sms_count INT,
    data_gb DOUBLE,
    usage_date DATE,
    circle STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
TBLPROPERTIES ('skip.header.line.count'='1');

LOAD DATA INPATH
'<<telecom>>/raw/<<customer_usage>>/<<telecom_usage.csv>>'
INTO TABLE <<telecom_usage_raw>>;

DROP TABLE IF EXISTS <<telecom_usage_final>>;

CREATE TABLE <<telecom_usage_final>> AS
SELECT
    usage_id,
    customer_id,

```

```

        plan,
        call_minutes,
        sms_count,
        data_gb,
        usage_date,
        circle,
        CASE
            WHEN data_gb >= 10 THEN 'HIGH'
            ELSE 'NORMAL'
        END AS usage_category
FROM <<telecom_usage_raw>>;
"
""",
)

```

4. LOAD DATA WAREHOUSE / VERIFY FINAL TABLE

```

load_dw = BashOperator(
    task_id="load_dw",
    bash_command="""
beeline -u jdbc:hive2://<<localhost>>:<<10000>>/<<default>> -e "

USE <<telecom_dw>>;

SELECT COUNT(*) AS raw_count
FROM <<telecom_usage_raw>>;

SELECT COUNT(*) AS final_count
FROM <<telecom_usage_final>>;

SELECT
    usage_id,
    customer_id,
    plan,
    data_gb,
    circle,
    usage_category
FROM <<telecom_usage_final>>;

SELECT
    usage_category,
    COUNT(*) AS count
FROM <<telecom_usage_final>>
GROUP BY usage_category;
"
""",
)

```

```
# DAG DEPENDENCIES
extract >> load_to_hdfs >> transform_hive >> load_dw
```

Group C: Airflow Execution & Web UI Verification

Terminal 3: Airflow Standalone / Scheduler Execution

1. Start Airflow Standalone Environment (if not already running):

Bash



```
airflow standalone
```

(Note the admin username and password printed in the terminal console).

Terminal 4: Trigger DAG via CLI (Optional / Direct)

1. Trigger DAG Execution:

Bash



```
airflow dags trigger <<telecom_customer_usage_etl>>
```

Web Browser: Airflow GUI

1. Open Airflow Dashboard:
Navigating to `http://<<localhost>>:<<8080>>`
2. Log In: Use the credentials obtained from Terminal 3.
3. Locate DAG: Find `<<telecom_customer_usage_etl>>` and unpause/trigger it.
TXT
4. Verify Graph Execution View:

Plaintext



```
extract --> load_to_hdfs --> transform_hive --> load_dw
```

Confirm that all 4 tasks run sequentially and complete with **Green (SUCCESS)** status.

TXT

Group D: Independent Result Verification

Terminal 5: Hive Shell Verification

1. Connect to Hive via Beeline Shell:

Bash



```
beeline -u jdbc:hive2://<<localhost>>:<<10000>>/<<default>>
```

1. Run Verification Queries:

SQL



```
USE <<telecom_dw>>;
```

```
-- 1. Check raw data persistence
```

```
SELECT * FROM <<telecom_usage_raw>> LIMIT 5;
```

```
-- 2. Verify total transformed records (Expected: 15)
```

```
SELECT COUNT(*) FROM <<telecom_usage_final>>;
```

```
-- 3. Verify category breakdown transformation logic
```

```
SELECT usage_category, COUNT(*) FROM <<telecom_usage_final>> GROUP BY usage_category;
```

```
-- 4. Sample check for transformation accuracy
```

```
SELECT usage_id, customer_id, data_gb, usage_category
```

```
FROM <<telecom_usage_final>>
```

```
WHERE data_gb >= 10;
```